The Complete Python Guide — Beginner to Production-Ready

> **Who this is for:** Absolute beginners who want to reach an intermediate level capable of answering enterprise/production Python interview questions confidently. **What's covered:** Core Python → OOP → Modules → File Handling → Error Handling → Comprehensions → Iterators → Decorators → Concurrency → Testing → Best Practices → Interview Q&A **Best Resources Referenced Throughout**

Top Resources (Bookmark These First)

Resource	Type	Best For
docs.python.org	Official Docs	Reference for everything
Real Python	Articles/Tutorials	Deep-dives with examples
Python Crash Course (book)	Book	Structured beginner path
Fluent Python (book)	Book	Intermediate/Advanced
CS50P – Harvard	Free Course	Hands-on beginner course
PyMOTW	Articles	Standard library deep dives
Exercism – Python Track	Practice	Hands-on exercises
Leetcode (Easy)	Practice	Interview prep (basic DSA)

Learning Roadmap

```
Phase 1: Basics (Week 1–2)
  → Variables, Data Types, Operators, Conditionals, Loops

Phase 2: Functions & Collections (Week 2–3)
  → Functions, Lists, Dicts, Sets, Tuples, Comprehensions

Phase 3: OOP & Modules (Week 3–4)
  → Classes, Inheritance, Modules, Packages, pip

Phase 4: File & Error Handling (Week 4–5)
  → File I/O, Exceptions, Context Managers
```

Phase 5: Pythonic Python (Week 5–6)
→ Iterators, Generators, Decorators, *args/**kwargs

Phase 6: Production Skills (Week 6–8)
→ Virtual Envs, Logging, Testing, Type Hints, Packaging

Phase 7: Concurrency (Week 8–9)
→ Threading, Multiprocessing, Async/Await

PHASE 1 — Python Basics

1.1 Setup

```bash
# Download Python from python.org
# Verify installation
python3 --version       # Python 3.11.x

# Use an IDE: VS Code (recommended) or PyCharm
# Install VS Code extension: "Python" by Microsoft
```

1.2 Your First Program

```python
# hello.py
print("Hello, World!")

# Run it:
# python3 hello.py
```

1.3 Variables & Data Types

Python is **dynamically typed** — you don't declare types explicitly.

```python
```

```python
# Basic types
name = "Alice"        # str
age = 30              # int
height = 5.7          # float
is_active = True      # bool
nothing = None        # NoneType

# Check type
print(type(name))     # <class 'str'>
print(type(age))      # <class 'int'>

# Type conversion
x = int("42")         # str → int
y = float("3.14")     # str → float
z = str(100)          # int → str
a = bool(0)           # 0 → False, anything else → True
```

1.4 Strings (Very Important!)

```python
```

```python
# String basics
s = "Hello, Python!"

# Slicing: s[start:end:step]
print(s[0:5])          # Hello
print(s[-1])           # ! (negative index = from end)
print(s[::-1])         # !nohtyP ,olleH (reverse)

# f-strings (modern, preferred way — Python 3.6+)
name = "Alice"
age = 30
message = f"My name is {name} and I am {age} years old."
print(message)

# Useful string methods
text = "  hello world  "
print(text.strip())        # "hello world"      — remove whitespace
print(text.upper())        # "  HELLO WORLD  "
print(text.lower())        # "  hello world  "
print("hello".replace("l", "r"))   # "herro"
print("a,b,c".split(","))          # ['a', 'b', 'c']
print(",".join(["a","b","c"]))     # "a,b,c"
print("hello world".startswith("hello"))   # True
print("hello".find("ll"))   # 2 (index where found)
print("hello".count("l"))   # 2

# Multi-line string
bio = """
This is a
multi-line string.
"""
```

1.5 Operators

```python
```

```python
# Arithmetic
print(10 + 3)   # 13
print(10 - 3)   # 7
print(10 * 3)   # 30
print(10 / 3)   # 3.333... (always float)
print(10 // 3)  # 3  (floor division — integer result)
print(10 % 3)   # 1  (modulo — remainder)
print(2 ** 8)   # 256 (power)

# Comparison (returns bool)
print(5 > 3)    # True
print(5 == 5)   # True
print(5 != 3)   # True
print(5 >= 5)   # True

# Logical
print(True and False)  # False
print(True or False)   # True
print(not True)        # False

# Identity and Membership
x = [1, 2, 3]
print(1 in x)          # True
print(4 not in x)      # True

a = None
print(a is None)       # True  ← use 'is' for None, not ==
print(a is not None)   # False
```

1.6 Conditionals

```python
```

```python
score = 75

if score >= 90:
    print("A")
elif score >= 75:
    print("B")
elif score >= 60:
    print("C")
else:
    print("Fail")

# One-liner (ternary)
grade = "Pass" if score >= 60 else "Fail"
print(grade)  # Pass

# match statement (Python 3.10+) — like switch/case
status = 404
match status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unknown")
```

1.7 Loops

```python
```

```python
# for loop
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 2):  # start, stop, step → 2,4,6,8
    print(i)

# Loop over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# enumerate — gives index + value
for index, fruit in enumerate(fruits):
    print(index, fruit)  # 0 apple, 1 banana...

# while loop
count = 0
while count < 5:
    print(count)
    count += 1

# Loop control
for i in range(10):
    if i == 3:
        continue   # skip this iteration
    if i == 7:
        break      # exit loop entirely
    print(i)

# else on loops — runs when loop completes normally (no break)
for i in range(5):
    print(i)
else:
    print("Loop finished without break")
```

PHASE 2 — Functions & Collections

2.1 Functions

```python
```

```python
# Basic function
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice"))   # Hello, Alice!

# Default parameters
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Bob"))             # Hello, Bob!
print(greet("Bob", "Hi"))       # Hi, Bob!

# Keyword arguments (order doesn't matter)
def describe(name, age, city):
    return f"{name}, {age}, from {city}"

print(describe(age=25, city="NY", name="Alice"))

# Multiple return values (returns a tuple)
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([3, 1, 9, 5])
print(low, high)   # 1 9

# *args — variable number of positional arguments
def add(*numbers):
    return sum(numbers)

print(add(1, 2, 3, 4))    # 10

# **kwargs — variable number of keyword arguments
def display(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

display(name="Alice", age=30, city="NY")

# Combining: (regular, *args, **kwargs)
def mixed(x, *args, **kwargs):
    print(x, args, kwargs)
```

```python
mixed(1, 2, 3, a=4, b=5)  # 1 (2, 3) {'a': 4, 'b': 5}
```

2.2 Lambda Functions

```python
# Short anonymous function: lambda arguments: expression
square = lambda x: x ** 2
print(square(5))   # 25

# Commonly used with map, filter, sorted
numbers = [3, 1, 4, 1, 5, 9, 2, 6]

# map — apply function to each element
squared = list(map(lambda x: x**2, numbers))

# filter — keep elements that pass a test
evens = list(filter(lambda x: x % 2 == 0, numbers))

# sorted with custom key
words = ["banana", "apple", "cherry", "date"]
sorted_by_length = sorted(words, key=lambda w: len(w))
print(sorted_by_length)  # ['date', 'apple', 'banana', 'cherry']
```

2.3 Lists

```python
```

```python
# Lists are ordered, mutable, allow duplicates
fruits = ["apple", "banana", "cherry"]

# Access
print(fruits[0])       # apple
print(fruits[-1])      # cherry
print(fruits[1:3])     # ['banana', 'cherry']

# Modify
fruits.append("date")          # add to end
fruits.insert(1, "avocado")    # insert at index
fruits.remove("banana")        # remove by value
popped = fruits.pop()          # remove and return last item
popped = fruits.pop(0)         # remove and return item at index

# Useful methods
numbers = [3, 1, 4, 1, 5, 9]
print(len(numbers))            # 6
print(sorted(numbers))         # [1, 1, 3, 4, 5, 9]  — returns new list
numbers.sort()                 # sorts in place
numbers.sort(reverse=True)     # descending
print(numbers.count(1))        # 2
print(numbers.index(4))        # finds first occurrence index
numbers.reverse()              # reverses in place
numbers2 = numbers.copy()      # shallow copy (important!)
numbers.extend([10, 11])       # add multiple items

# Check membership
print("apple" in fruits)       # True
```

2.4 Dictionaries (Most Important Data Structure!)

```python
```

```python
# Dicts are key-value pairs, ordered (Python 3.7+), mutable
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Access
print(person["name"])          # Alice
print(person.get("name"))      # Alice (safe — returns None if missing)
print(person.get("email", "N/A"))  # N/A (default if missing)

# Modify
person["age"] = 31             # update
person["email"] = "a@b.com"    # add new key
del person["city"]             # delete key
removed = person.pop("email")  # remove and return

# Iteration
for key in person:
    print(key)

for key, value in person.items():
    print(f"{key}: {value}")

for value in person.values():
    print(value)

# Useful methods
print(person.keys())           # dict_keys(['name', 'age'])
print(person.values())         # dict_values(['Alice', 31])
print("name" in person)        # True (checks keys)

# Merge dicts (Python 3.9+)
a = {"x": 1}
b = {"y": 2}
merged = a | b                 # {'x': 1, 'y': 2}
a |= b                         # update a with b in place

# setdefault — set key only if it doesn't exist
person.setdefault("country", "US")

# defaultdict from collections — auto-creates missing keys
```

```python
from collections import defaultdict
dd = defaultdict(list)
dd["fruits"].append("apple")   # no KeyError!
dd["fruits"].append("banana")
print(dd)  # {'fruits': ['apple', 'banana']}

# Counter — count occurrences
from collections import Counter
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
c = Counter(words)
print(c)                    # Counter({'apple': 3, 'banana': 2, 'cherry': 1})
print(c.most_common(2))        # [('apple', 3), ('banana', 2)]
```

2.5 Tuples

```python
# Tuples are ordered, IMMUTABLE (can't change after creation)
point = (3, 4)
rgb = (255, 128, 0)

# Unpacking
x, y = point
r, g, b = rgb

# Single element tuple (note the comma!)
single = (42,)   # without comma it's just (42) which is an int

# Named tuples — like lightweight classes
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p.x, p.y)    # 3 4
print(p)          # Point(x=3, y=4)

# When to use tuples vs lists:
# - Tuple: fixed data, function return values, dict keys (hashable)
# - List: data that changes, when you need .append()/.remove()
```

2.6 Sets

```python
```

```python
# Sets are unordered, unique elements, mutable
fruits = {"apple", "banana", "cherry", "apple"}  # duplicate removed
print(fruits)   # {'apple', 'banana', 'cherry'}

# Add / Remove
fruits.add("date")
fruits.discard("banana")   # safe remove (no error if missing)
fruits.remove("cherry")    # raises KeyError if missing

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)   # {1,2,3,4,5,6}  — union
print(a & b)   # {3, 4}         — intersection
print(a - b)   # {1, 2}         — difference (in a but not b)
print(a ^ b)   # {1,2,5,6}      — symmetric difference

# Use case: fast membership check
allowed = {"admin", "editor", "viewer"}
role = "admin"
print(role in allowed)   # True — O(1) lookup, much faster than list!

# Deduplicate a list
nums = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(nums))
```

2.7 List Comprehensions

```python
```

```python
# Basic syntax: [expression for item in iterable if condition]

# Without comprehension:
squares = []
for x in range(10):
    squares.append(x**2)

# With comprehension:
squares = [x**2 for x in range(10)]

# With condition
even_squares = [x**2 for x in range(10) if x % 2 == 0]
print(even_squares)  # [0, 4, 16, 36, 64]

# Dict comprehension
words = ["hello", "world", "python"]
lengths = {word: len(word) for word in words}
print(lengths)  # {'hello': 5, 'world': 5, 'python': 6}

# Set comprehension
unique_lengths = {len(word) for word in words}

# Nested comprehension (flattening a 2D list)
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
print(flat)  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

PHASE 3 — OOP (Object-Oriented Programming)

3.1 Classes & Objects

```python
```

```python
# A class is a blueprint; an object is an instance
class Dog:
    # Class variable — shared by ALL instances
    species = "Canis lupus familiaris"

    # __init__ is the constructor — called when object is created
    def __init__(self, name, age):
        # Instance variables — unique to each object
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        return f"{self.name} says: Woof!"

    def __repr__(self):
        # Developer-friendly string representation
        return f"Dog(name={self.name!r}, age={self.age})"

    def __str__(self):
        # User-friendly string representation
        return f"{self.name} ({self.age} years old)"


# Create instances
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.name)        # Buddy
print(dog1.bark())      # Buddy says: Woof!
print(dog1)             # Buddy (3 years old) — uses __str__
print(repr(dog1))       # Dog(name='Buddy', age=3) — uses __repr__
print(dog1.species)     # Canis lupus familiaris
print(Dog.species)      # Same — accessed from class or instance
```

3.2 Inheritance

```python
```

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclass must implement speak()")

    def __str__(self):
        return f"{self.__class__.__name__}: {self.name}"


class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"


class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"


class GoldenRetriever(Dog):
    def speak(self):
        # super() calls parent class method
        base = super().speak()
        return f"{base} (and wags tail)"


# Polymorphism — same interface, different behavior
animals = [Dog("Buddy"), Cat("Whiskers"), GoldenRetriever("Charlie")]
for animal in animals:
    print(animal.speak())

# isinstance & issubclass
print(isinstance(animals[0], Dog))      # True
print(isinstance(animals[0], Animal))   # True
print(issubclass(Dog, Animal))          # True
```

3.3 Properties, Class Methods & Static Methods

```python
```

```python
class Circle:
    PI = 3.14159

    def __init__(self, radius):
        self._radius = radius   # _radius = "protected" (convention)

    # @property — getter, makes attribute read-only or computed
    @property
    def radius(self):
        return self._radius

    # @setter — allows controlled assignment
    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

    @property
    def area(self):
        return Circle.PI * self._radius ** 2

    # @classmethod — receives class (cls) not instance (self)
    # Used for alternative constructors
    @classmethod
    def from_diameter(cls, diameter):
        return cls(diameter / 2)

    # @staticmethod — no access to class or instance
    # Just a utility function grouped with the class
    @staticmethod
    def is_valid_radius(value):
        return value >= 0


c = Circle(5)
print(c.radius)        # 5
print(c.area)          # 78.53975
c.radius = 10          # uses setter
# c.radius = -1        # raises ValueError

c2 = Circle.from_diameter(20)  # classmethod
print(c2.radius)       # 10.0
```

```
print(Circle.is_valid_radius(-1))  # False
```

3.4 Dunder (Magic) Methods

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

    def __len__(self):
        return 2    # number of dimensions

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __abs__(self):
        return (self.x**2 + self.y**2) ** 0.5


v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)      # Vector(4, 6)
print(v1 * 3)       # Vector(3, 6)
print(abs(v2))      # 5.0
print(len(v1))      # 2
print(v1 == v2)     # False
```

3.5 Dataclasses (Python 3.7+ — very common in production)

```python
```

```python
from dataclasses import dataclass, field
from typing import List

@dataclass
class Employee:
    name: str
    department: str
    salary: float = 50000.0
    skills: List[str] = field(default_factory=list)  # mutable default must use field()

    def give_raise(self, percent):
        self.salary *= (1 + percent / 100)


e = Employee("Alice", "Engineering", 80000)
e.skills.append("Python")
print(e)          # Employee(name='Alice', department='Engineering', salary=80000, skills=['Python'])
e.give_raise(10)
print(e.salary)     # 88000.0

# @dataclass automatically generates __init__, __repr__, __eq__
# frozen=True makes it immutable (like a named tuple with types)
@dataclass(frozen=True)
class Point:
    x: float
    y: float
```

PHASE 4 — Modules, Packages & File Handling

4.1 Modules & Packages

```python
```

```python
# A module = a .py file
# A package = a folder with __init__.py

# math module
import math
print(math.sqrt(16))     # 4.0
print(math.pi)           # 3.14159...
print(math.ceil(4.2))    # 5
print(math.floor(4.9))   # 4

# Import specific items
from math import sqrt, pi
print(sqrt(25))          # 5.0

# Alias imports
import numpy as np       # third-party lib

# os module — interact with operating system
import os
print(os.getcwd())               # current working directory
print(os.path.exists("file.txt"))  # check if file exists
print(os.path.join("folder", "file.txt"))  # OS-safe path
os.makedirs("new_folder", exist_ok=True)   # create directory
files = os.listdir(".")          # list files in current dir

# sys module
import sys
print(sys.version)               # Python version
print(sys.argv)                  # command-line arguments
sys.exit(0)                      # exit program (0 = success)

# random module
import random
print(random.randint(1, 10))     # random int between 1–10
print(random.choice([1, 2, 3]))  # random element from list
random.shuffle([1, 2, 3, 4])     # shuffle list in place
print(random.random())           # float between 0.0 and 1.0

# datetime module
from datetime import datetime, date, timedelta
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))   # format date
today = date.today()
```

```python
tomorrow = today + timedelta(days=1)
print(tomorrow)

# json module — very common in APIs
import json
data = {"name": "Alice", "scores": [95, 87, 92]}

# dict → JSON string
json_str = json.dumps(data, indent=2)
print(json_str)

# JSON string → dict
parsed = json.loads(json_str)
print(parsed["name"])

# write to file
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

# read from file
with open("data.json", "r") as f:
    loaded = json.load(f)
```

4.2 File Handling

```python
```

```python
# ALWAYS use 'with' statement — auto-closes file even on error

# Write to file
with open("example.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("Second line\n")

# Append to file
with open("example.txt", "a") as f:
    f.write("Third line\n")

# Read entire file
with open("example.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line (memory-efficient for large files)
with open("example.txt", "r") as f:
    for line in f:
        print(line.strip())

# Read all lines into a list
with open("example.txt", "r") as f:
    lines = f.readlines()

# Working with CSV
import csv

# Write CSV
with open("data.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerow({"name": "Alice", "age": 30})
    writer.writerow({"name": "Bob", "age": 25})

# Read CSV
with open("data.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)   # {'name': 'Alice', 'age': '30'}
```

4.3 pathlib (Modern File Paths — Preferred Over os.path)

```python
from pathlib import Path

# Create path objects
p = Path(".")            # current directory
home = Path.home()       # home directory

# Path operations
config = Path("config") / "settings.json"   # / operator joins paths
print(config)            # config/settings.json

# File info
p = Path("example.txt")
print(p.exists())        # True/False
print(p.name)            # "example.txt"
print(p.stem)            # "example" (no extension)
print(p.suffix)          # ".txt"
print(p.parent)          # parent directory

# Read/write
p.write_text("Hello from pathlib!")
content = p.read_text()

# List directory contents
for file in Path(".").iterdir():
    print(file)

# Find files by pattern
for py_file in Path(".").rglob("*.py"):
    print(py_file)
```

PHASE 5 — Error Handling

5.1 Try / Except / Finally

```python
```

```python
# Basic try-except
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")

# Catch multiple exceptions
try:
    data = int(input("Enter a number: "))
    result = 10 / data
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Can't divide by zero!")
except (TypeError, AttributeError) as e:
    print(f"Type or attribute error: {e}")
except Exception as e:
    # Catch-all — be careful, can hide bugs
    print(f"Unexpected error: {e}")
else:
    # Runs ONLY if no exception occurred
    print(f"Result: {result}")
finally:
    # ALWAYS runs, exception or not — great for cleanup
    print("Execution complete")

# Re-raise exception
try:
    risky_operation()
except ValueError as e:
    print("Logging error...")
    raise   # re-raises the same exception
```

5.2 Custom Exceptions

```python
```

```python
# Custom exceptions are important in production code
class AppError(Exception):
    """Base exception for this application."""
    pass

class ValidationError(AppError):
    """Raised when input validation fails."""
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(f"Validation error on '{field}': {message}")

class DatabaseError(AppError):
    """Raised when database operations fail."""
    pass


# Usage
def validate_age(age):
    if not isinstance(age, int):
        raise ValidationError("age", "Must be an integer")
    if age < 0 or age > 150:
        raise ValidationError("age", "Must be between 0 and 150")
    return True


try:
    validate_age(-5)
except ValidationError as e:
    print(e.field)     # age
    print(e.message)   # Must be between 0 and 150
    print(str(e))      # Validation error on 'age': Must be between 0 and 150
```

5.3 Context Managers

```python
```

```python
# Context managers handle setup and teardown with 'with'
# You already saw them with files — here's how to create your own

# Method 1: Using a class with __enter__ and __exit__
class DatabaseConnection:
    def __init__(self, db_url):
        self.db_url = db_url
        self.connection = None

    def __enter__(self):
        print(f"Connecting to {self.db_url}...")
        self.connection = f"<connection to {self.db_url}>"
        return self.connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Closing connection...")
        self.connection = None
        # Return True to suppress exceptions; False/None to propagate
        return False


with DatabaseConnection("postgresql://localhost/mydb") as conn:
    print(f"Using: {conn}")
    # connection automatically closed after this block


# Method 2: Using contextlib.contextmanager (simpler!)
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    try:
        yield    # code in 'with' block runs here
    finally:
        elapsed = time.time() - start
        print(f"Time elapsed: {elapsed:.4f}s")


with timer():
```

```
# some slow operation
sum(range(1_000_000))
```

PHASE 5B — Pythonic Python

6.1 Iterators & Generators

```python
```

```python
# Iterator: any object with __iter__ and __next__ methods
# Iterable: any object you can loop over (list, dict, str...)

# range is lazy — doesn't create all numbers in memory
r = range(1_000_000)   # no memory used until you iterate

# zip — combine multiple iterables
names = ["Alice", "Bob", "Charlie"]
scores = [95, 87, 72]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# zip with unequal lengths
for a, b in zip([1,2,3], [4,5]):
    print(a, b)   # stops at shortest: (1,4), (2,5)

# itertools — extremely useful
import itertools
print(list(itertools.chain([1,2], [3,4], [5])))  # [1,2,3,4,5]
print(list(itertools.islice(range(100), 5)))       # [0,1,2,3,4]
for combo in itertools.combinations([1,2,3], 2):
    print(combo)  # (1,2),(1,3),(2,3)

# GENERATORS — memory-efficient lazy sequences
# Use 'yield' instead of 'return'
def count_up(start, end):
    current = start
    while current <= end:
        yield current   # pause, return value, then resume
        current += 1

gen = count_up(1, 5)
print(next(gen))   # 1
print(next(gen))   # 2
for num in gen:    # continues from where we left off
    print(num)     # 3, 4, 5

# Generator expression (like list comprehension but lazy)
squares_gen = (x**2 for x in range(1_000_000))  # no memory used!
print(next(squares_gen))    # 0
print(next(squares_gen))    # 1

# Real use case: reading a large file line by line
```

```python
def read_large_file(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()

# Process without loading whole file in memory
for line in read_large_file("huge_data.txt"):
    process(line)
```

6.2 Decorators

```python
```

```python
# A decorator wraps a function to add behavior
# They are functions that take a function and return a new function

# Basic decorator
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function")
        result = func(*args, **kwargs)
        print("After function")
        return result
    return wrapper

@my_decorator
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Alice")
# Before function
# Hello, Alice!
# After function

# Important: always use functools.wraps to preserve metadata
from functools import wraps

def timer(func):
    import time
    @wraps(func)    # preserves __name__, __doc__ of original function
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f}s")
        return result
    return wrapper

# Decorator with arguments
def retry(max_attempts=3):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
```

```python
            except Exception as e:
                if attempt == max_attempts:
                    raise
                print(f"Attempt {attempt} failed: {e}. Retrying...")
        return wrapper
    return decorator

@retry(max_attempts=3)
def unstable_api_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("API unreachable")
    return "Success!"

# Common built-in decorators:
# @property, @classmethod, @staticmethod
# @functools.lru_cache — memoization (caching results)
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(100))  # instant — cached results!
```

6.3 Type Hints (Essential in Production)

```python
```

```python
# Type hints don't affect runtime — they're documentation + IDE support
# Use mypy to check types statically

def greet(name: str) -> str:
    return f"Hello, {name}!"

def add(a: int, b: int) -> int:
    return a + b

# Common type hints
from typing import List, Dict, Tuple, Optional, Union, Any, Callable

def process_scores(scores: List[float]) -> Dict[str, float]:
    return {
        "min": min(scores),
        "max": max(scores),
        "avg": sum(scores) / len(scores)
    }

# Optional = can be None
def find_user(user_id: int) -> Optional[Dict]:
    # returns dict or None
    pass

# Union = one of multiple types
def stringify(value: Union[int, float, str]) -> str:
    return str(value)

# Python 3.10+ — simpler union syntax
def newer(value: int | float | str | None) -> str:
    return str(value) if value else "empty"

# Callable type
from typing import Callable
def apply(func: Callable[[int], int], value: int) -> int:
    return func(value)

# TypedDict — for dict with known keys
from typing import TypedDict
class UserProfile(TypedDict):
    name: str
    age: int
    email: str
```

```
# Type aliases
Vector = List[float]
def scale(scalar: float, vector: Vector) -> Vector:
    return [scalar * num for num in vector]
```

PHASE 6 — Production Python

7.1 Virtual Environments & Dependency Management

```bash
# Virtual environments isolate project dependencies

# Create venv
python3 -m venv venv

# Activate
source venv/bin/activate        # macOS/Linux
venv\Scripts\activate           # Windows

# Deactivate
deactivate

# Install packages
pip install requests flask sqlalchemy

# Save dependencies
pip freeze > requirements.txt

# Install from requirements.txt
pip install -r requirements.txt

# Modern alternative: Poetry (recommended for new projects)
pip install poetry
poetry new my-project
poetry add requests
poetry install
```

7.2 Logging (Never Use print() in Production)

```python
```

```python
import logging

# Basic setup
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("app.log"),
        logging.StreamHandler()        # also print to console
    ]
)

logger = logging.getLogger(__name__)   # best practice: use module name

# Log levels (in order of severity)
logger.debug("Debug details for developers")     # most verbose
logger.info("Application started successfully")
logger.warning("Config file not found, using defaults")
logger.error("Database connection failed")
logger.critical("System is shutting down!")      # most severe

# Structured logging for production (log as JSON)
import json

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            "timestamp": self.formatTime(record),
            "level": record.levelname,
            "message": record.getMessage(),
            "module": record.module,
        }
        return json.dumps(log_data)

# Exception logging
try:
    risky()
except Exception:
    logger.exception("Error in risky()")  # logs traceback automatically!

# Best practices:
# 1. Never log sensitive data (passwords, tokens, PII)
# 2. Use structured logging (JSON) for log aggregation tools
```

```
# 3. Use named loggers (getLogger(__name__))
# 4. Set appropriate log levels per environment (DEBUG in dev, INFO/WARNING in prod)
```

7.3 Environment Variables & Configuration

```python
import os
from dotenv import load_dotenv   # pip install python-dotenv

# .env file (NEVER commit to git):
# DATABASE_URL=postgresql://user:pass@localhost/mydb
# SECRET_KEY=my-super-secret-key
# DEBUG=true

load_dotenv()   # loads .env into environment

DB_URL = os.getenv("DATABASE_URL", "sqlite:///default.db")   # with default
SECRET_KEY = os.environ["SECRET_KEY"]   # raises KeyError if missing

# Structured config using dataclass
from dataclasses import dataclass

@dataclass
class Config:
    db_url: str = os.getenv("DATABASE_URL", "sqlite:///app.db")
    debug: bool = os.getenv("DEBUG", "false").lower() == "true"
    secret_key: str = os.getenv("SECRET_KEY", "changeme")
    max_connections: int = int(os.getenv("MAX_CONNECTIONS", "10"))

config = Config()
print(config.db_url)
```

7.4 Testing with pytest

```bash
pip install pytest pytest-cov
```

```python
```

```python
# app/calculator.py
def add(a, b):
    return a + b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

# tests/test_calculator.py
import pytest
from app.calculator import add, divide

# Basic test
def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0

# Test exceptions
def test_divide_by_zero():
    with pytest.raises(ValueError, match="Cannot divide by zero"):
        divide(10, 0)

# Parametrize — run same test with multiple inputs
@pytest.mark.parametrize("a, b, expected", [
    (10, 2, 5.0),
    (9, 3, 3.0),
    (-6, 2, -3.0),
])
def test_divide(a, b, expected):
    assert divide(a, b) == expected

# Fixtures — reusable test setup
@pytest.fixture
def sample_users():
    return [
        {"id": 1, "name": "Alice"},
        {"id": 2, "name": "Bob"},
    ]

def test_user_count(sample_users):
    assert len(sample_users) == 2
```

```python
# Mocking (isolate from real dependencies)
from unittest.mock import patch, MagicMock

def test_api_call():
    with patch("requests.get") as mock_get:
        mock_get.return_value.json.return_value = {"status": "ok"}
        mock_get.return_value.status_code = 200

        result = your_function_that_calls_api()
        assert result == {"status": "ok"}
        mock_get.assert_called_once()
```

```bash
# Run tests
pytest                      # all tests
pytest tests/test_calculator.py # specific file
pytest -v                   # verbose
pytest --cov=app            # coverage report
pytest -k "test_divide"     # run tests matching name
```

7.5 Packaging & Project Structure

```
my_project/
├── src/
│   └── my_package/
│       ├── __init__.py
│       ├── core.py
│       └── utils.py
├── tests/
│   ├── __init__.py
│   ├── test_core.py
│   └── test_utils.py
├── .env
├── .gitignore
├── pyproject.toml        # modern (Poetry/PEP 517)
├── requirements.txt
└── README.md
```

```toml
```

```
# pyproject.toml
[tool.poetry]
name = "my-package"
version = "0.1.0"
description = "A sample Python package"
authors = ["Your Name <you@example.com>"]

[tool.poetry.dependencies]
python = "^3.11"
requests = "^2.28"

[tool.poetry.dev-dependencies]
pytest = "^7.0"
mypy = "^1.0"
black = "^23.0"
```

7.6 Code Quality Tools

```bash
```

```
# Black — opinionated code formatter
pip install black
black .                  # format all Python files

# flake8 — linting
pip install flake8
flake8 src/              # check for style issues

# mypy — static type checking
pip install mypy
mypy src/                # type-check

# isort — sort imports
pip install isort
isort .

# pre-commit — run checks before every commit
pip install pre-commit
# .pre-commit-config.yaml defines which checks to run
pre-commit install
pre-commit run --all-files

# ruff — modern, fast linter (replaces flake8 + isort + more)
pip install ruff
ruff check .
ruff format .
```

PHASE 7 — Concurrency

8.1 The GIL (Global Interpreter Lock)

```
The GIL is a mutex in CPython that allows only ONE thread to execute
Python bytecode at a time. This means:

- Threading: Good for I/O-bound tasks (network calls, file I/O)
  → Threads wait for I/O, GIL is released during wait

- Multiprocessing: Good for CPU-bound tasks (data processing, computation)
  → Each process has its own GIL + memory space

- asyncio: Good for high-volume I/O-bound tasks (async/await)
```

→ Single-threaded, cooperative multitasking

Rule of thumb:
I/O-bound + few tasks → threading
I/O-bound + many tasks → asyncio
CPU-bound → multiprocessing

8.2 Threading

python

```python
import threading
import time

def download(url, results, index):
    print(f"Downloading {url}...")
    time.sleep(2)  # simulate download
    results[index] = f"Data from {url}"

# Without threading: takes 6 seconds (3 sequential downloads)
# With threading: takes ~2 seconds (parallel)
urls = ["url1", "url2", "url3"]
results = [None] * len(urls)
threads = []

for i, url in enumerate(urls):
    t = threading.Thread(target=download, args=(url, results, i))
    threads.append(t)
    t.start()

for t in threads:
    t.join()  # wait for all threads to finish

print(results)

# ThreadPoolExecutor — cleaner way
from concurrent.futures import ThreadPoolExecutor

def fetch(url):
    time.sleep(1)
    return f"result from {url}"

with ThreadPoolExecutor(max_workers=5) as executor:
    futures = [executor.submit(fetch, url) for url in urls]
    results = [f.result() for f in futures]

# Thread safety — use locks to prevent race conditions
counter = 0
lock = threading.Lock()

def increment():
    global counter
```

```
    with lock:      # only one thread can enter at a time
        counter += 1
```

8.3 Multiprocessing

```python
from concurrent.futures import ProcessPoolExecutor
import multiprocessing

def cpu_heavy(n):
    """CPU-bound: compute sum of squares"""
    return sum(i**2 for i in range(n))

data = [1_000_000, 2_000_000, 3_000_000, 4_000_000]

# Single process (slow for CPU work)
results = [cpu_heavy(n) for n in data]

# Multiple processes (uses all CPU cores)
with ProcessPoolExecutor(max_workers=multiprocessing.cpu_count()) as ex:
    results = list(ex.map(cpu_heavy, data))

print(results)
```

8.4 Async / Await (asyncio)

```python
```

```python
import asyncio
import aiohttp   # pip install aiohttp

# async def = coroutine function
async def fetch_url(session, url):
    async with session.get(url) as response:
        return await response.text()

async def main():
    urls = [
        "https://api.example.com/users/1",
        "https://api.example.com/users/2",
        "https://api.example.com/users/3",
    ]

    async with aiohttp.ClientSession() as session:
        # gather runs coroutines concurrently
        results = await asyncio.gather(
            *[fetch_url(session, url) for url in urls]
        )

    return results

# Run the event loop
results = asyncio.run(main())

# Async context manager
class AsyncDB:
    async def __aenter__(self):
        self.conn = await connect_to_db()
        return self.conn

    async def __aexit__(self, *args):
        await self.conn.close()

async def use_db():
    async with AsyncDB() as conn:
        data = await conn.query("SELECT * FROM users")

# Async generator
async def async_range(start, end):
    for i in range(start, end):
        await asyncio.sleep(0)   # yield control
```

```
        yield i

async def consume():
    async for num in async_range(0, 5):
        print(num)
```

PHASE 8 — Important Standard Library Modules

9.1 collections

python

```python
from collections import Counter, defaultdict, OrderedDict, deque, namedtuple

# Counter — count elements
votes = ["Alice", "Bob", "Alice", "Alice", "Bob", "Charlie"]
c = Counter(votes)
print(c.most_common(2))   # [('Alice', 3), ('Bob', 2)]
c1 = Counter("aabbcc")
c2 = Counter("abcd")
print(c1 + c2)          # Counter({'a': 3, 'b': 3, 'c': 3, 'd': 1})

# defaultdict — dict with default values
word_list = ["the", "quick", "brown", "fox", "the", "quick"]
word_count = defaultdict(int)
for word in word_list:
    word_count[word] += 1   # no KeyError!

# Group items by category
grouped = defaultdict(list)
data = [("fruit", "apple"), ("veggie", "carrot"), ("fruit", "banana")]
for category, item in data:
    grouped[category].append(item)
print(dict(grouped))   # {'fruit': ['apple', 'banana'], 'veggie': ['carrot']}

# deque — double-ended queue (O(1) append/pop from both ends)
dq = deque([1, 2, 3])
dq.appendleft(0)    # [0, 1, 2, 3]
dq.append(4)        # [0, 1, 2, 3, 4]
dq.popleft()        # removes 0
dq.pop()            # removes 4

# maxlen makes it a fixed-size sliding window
recent = deque(maxlen=3)
for i in range(6):
    recent.append(i)
print(recent)   # deque([3, 4, 5], maxlen=3)
```

9.2 functools

python

```python
from functools import reduce, partial, lru_cache, wraps

# reduce — apply function cumulatively
from functools import reduce
total = reduce(lambda x, y: x + y, [1, 2, 3, 4, 5])   # 15
product = reduce(lambda x, y: x * y, [1, 2, 3, 4, 5])  # 120

# partial — fix some arguments of a function
def power(base, exponent):
    return base ** exponent

square = partial(power, exponent=2)
cube = partial(power, exponent=3)
print(square(5))   # 25
print(cube(3))     # 27

# lru_cache — memoize (cache) results
@lru_cache(maxsize=None)
def expensive_computation(n):
    time.sleep(1)   # simulate expensive work
    return n * 2

print(expensive_computation(5))   # takes 1 second
print(expensive_computation(5))   # instant (cached)
print(expensive_computation.cache_info())   # hits, misses, maxsize, currsize
```

9.3 re (Regular Expressions)

```python
```

```python
import re

# Search for pattern
text = "Contact: alice@company.com or bob@example.org"
emails = re.findall(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b", text)
print(emails)   # ['alice@company.com', 'bob@example.org']

# Match — checks start of string
# Search — scans entire string
if re.search(r"\d+", "I have 42 apples"):
    print("Found a number!")

# Sub — replace
cleaned = re.sub(r"\s+", " ", "too  many   spaces")
print(cleaned)   # "too many spaces"

# Groups
match = re.search(r"(\d{4})-(\d{2})-(\d{2})", "Date: 2024-01-15")
if match:
    year, month, day = match.groups()
    print(year, month, day)   # 2024 01 15

# Compiled pattern (reuse for efficiency)
phone_pattern = re.compile(r"\b\d{3}[-.]?\d{3}[-.]?\d{4}\b")
numbers = phone_pattern.findall("Call 555-1234 or 555.5678")
```

PHASE 9 — Common Design Patterns

10.1 Singleton Pattern

```python
```

```python
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
print(s1 is s2)   # True — same instance!

# Use case: config object, database connection pool, logger
```

10.2 Factory Pattern

```python
class Dog:
    def speak(self): return "Woof"

class Cat:
    def speak(self): return "Meow"

class Bird:
    def speak(self): return "Tweet"

def animal_factory(animal_type: str):
    animals = {"dog": Dog, "cat": Cat, "bird": Bird}
    cls = animals.get(animal_type.lower())
    if not cls:
        raise ValueError(f"Unknown animal: {animal_type}")
    return cls()

pet = animal_factory("dog")
print(pet.speak())   # Woof
```

10.3 Observer Pattern

```python
```

```python
from typing import List, Callable

class EventEmitter:
    def __init__(self):
        self._listeners: dict[str, List[Callable]] = {}

    def on(self, event: str, callback: Callable):
        self._listeners.setdefault(event, []).append(callback)

    def emit(self, event: str, *args, **kwargs):
        for callback in self._listeners.get(event, []):
            callback(*args, **kwargs)

emitter = EventEmitter()
emitter.on("data_received", lambda d: print(f"Processing: {d}"))
emitter.on("data_received", lambda d: print(f"Logging: {d}"))
emitter.emit("data_received", {"user": "Alice"})
```

PHASE 10 — Basic DSA in Python

11.1 Big O — What It Is

```
Big O describes how performance scales with input size.

O(1)      — constant time: dict lookup, list index access
O(log n)  — binary search
O(n)      — linear: looping through a list
O(n log n)— sorting
O(n²)     — nested loops

Why it matters: If your function is O(n²) and n=10,000,
  that's 100,000,000 operations. Could be very slow!

Python data structure complexity:
  list:
    append/pop: O(1) amortized
    insert/delete at position: O(n)
    search (in): O(n)

  dict/set:
    lookup/insert/delete: O(1) average
```

```
deque:
  appendleft/popleft: O(1)   ← use this instead of list.insert(0, x)!
```

11.2 Stack & Queue

```python
# Stack (LIFO) — using a list
stack = []
stack.append(1)     # push
stack.append(2)
stack.append(3)
print(stack.pop())  # 3 — last in, first out

# Queue (FIFO) — use deque, NOT list (list.pop(0) is O(n)!)
from collections import deque

queue = deque()
queue.append("first")
queue.append("second")
queue.append("third")
print(queue.popleft())   # "first" — first in, first out
```

11.3 Two Pointers & Sliding Window

```python
```

```python
# Two pointers — check if array has pair summing to target
def has_pair_with_sum(sorted_nums, target):
    left, right = 0, len(sorted_nums) - 1
    while left < right:
        current = sorted_nums[left] + sorted_nums[right]
        if current == target:
            return True
        elif current < target:
            left += 1
        else:
            right -= 1
    return False

print(has_pair_with_sum([1, 3, 5, 7, 9], 10))   # True (1+9 or 3+7)

# Sliding window — max sum of k consecutive elements
def max_subarray_sum(nums, k):
    window_sum = sum(nums[:k])
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum

print(max_subarray_sum([2, 1, 5, 1, 3, 2], 3))   # 9 (5+1+3)
```

🎯 Top Python Interview Questions & Answers

Core Python

Q: What is the difference between `==` and `is`?

> `==` compares values (are they equal?). `is` compares identity (are they the same object in memory?). Always use `is` for `None` checks: `if x is None`.

Q: What are mutable vs immutable types?

> Mutable: list, dict, set — can be changed after creation. Immutable: int, float, str, tuple, frozenset — cannot be changed. Immutable objects are hashable (can be dict keys or set elements).

Q: What is the difference between `deepcopy` and `copy`?

```python
```

```python
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)   # new list, same inner lists
deep = copy.deepcopy(original)  # new list AND new inner lists

original[0].append(99)
print(shallow[0])   # [1, 2, 99] — affected!
print(deep[0])      # [1, 2] — not affected
```

Q: How does Python manage memory?

> Python uses reference counting + garbage collection (cyclic GC). Each object has a reference count; when it hits 0, memory is freed. The `gc` module handles circular references.

Q: What is a Python decorator and when would you use one?

> A decorator is a function that wraps another function to add behavior without modifying it. Common uses: logging, authentication checks, caching (lru_cache), retry logic, timing, input validation.

Q: What are `*args` and `**kwargs`?

> `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They allow flexible function signatures.

Q: What is a generator? Why use it over a list?

> A generator produces values lazily (one at a time) using `yield`. Unlike a list, it doesn't store all values in memory. Use it when data is large or infinite (e.g., reading a huge file line by line).

Q: What are list comprehensions vs generator expressions?

> List comprehension `[x for x in range(n)]` builds the whole list in memory. Generator expression `(x for x in range(n))` produces values lazily. Use generators when you only iterate once or when data is large.

Q: Explain `__init__` vs `__new__`.

> `__new__` creates the object (allocates memory). `__init__` initializes it (sets attributes). You rarely override `__new__` — only for immutable types or the Singleton pattern.

Q: What is MRO (Method Resolution Order)?

```python
```

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
# Python uses C3 linearization algorithm (left-to-right, depth-first, no repeat)
```

Q: What is the difference between `@classmethod` and `@staticmethod`?

> `@classmethod` receives the class (cls) as first argument — used for alternative constructors. `@staticmethod` receives nothing special — just a utility function namespaced inside a class.

Production & Enterprise

Q: How do you avoid blocking in an async application?

> Never call blocking code (file I/O, `time.sleep`, CPU-heavy work) in an async function without wrapping it. Use `await asyncio.sleep()`, `aiofiles` for async file I/O, `run_in_executor()` for blocking code.

Q: What is the GIL and how does it affect multithreading?

> The Global Interpreter Lock allows only one thread to run Python bytecode at once. For I/O-bound tasks, threads work fine (GIL is released during I/O waits). For CPU-bound tasks, use `multiprocessing` to bypass the GIL.

Q: How do you structure a large Python project?

> Use the `src/` layout, separate packages by domain (users/, orders/, etc.), a dedicated `tests/` folder, `pyproject.toml` for dependencies, environment variables for config (python-dotenv), type hints, and a linter + formatter in CI.

Q: What's the difference between `requirements.txt` and `pyproject.toml`?

> `requirements.txt` pins exact versions for deployment. `pyproject.toml` (PEP 517/518) is the modern standard that specifies flexible version ranges, build system, dev dependencies, and tool configuration in one file.

Q: How do you handle configuration in production?

> Use environment variables (never hardcode secrets). Load them with `python-dotenv` in development. In production, use platform secrets (AWS Secrets Manager, Vault, Kubernetes secrets). Use a config class to centralize and validate settings.

Q: How do you prevent SQL injection in Python?

> Never use string formatting to build SQL queries. Always use parameterized queries:

```python
# BAD
cursor.execute(f"SELECT * FROM users WHERE name = '{name}'")

# GOOD
cursor.execute("SELECT * FROM users WHERE name = ?", (name,))
# Or with ORM (SQLAlchemy):
User.query.filter_by(name=name).all()
```

Q: What's the best way to log in production?

> Use named loggers (`logging.getLogger(__name__)`), log in JSON format for log aggregation tools (ELK, Datadog, CloudWatch), never log sensitive data, use appropriate levels (DEBUG only in dev), and log exceptions with `logger.exception()` to get tracebacks.

Q: What are context managers and why are they important?

> Context managers (using `with`) guarantee cleanup code runs even if an exception occurs. Critical for resource management: file handles, database connections, locks. Implement via `__enter__`/`__exit__` or `@contextmanager`.

Q: How do you test code that calls external APIs?

> Mock the HTTP calls using `unittest.mock.patch` or libraries like `responses` or `pytest-httpx`. Never call real APIs in unit tests — tests should be fast, isolated, and deterministic.

Q: What is the difference between unit, integration, and end-to-end tests?

> Unit tests: test a single function/class in isolation (fast, use mocks). Integration tests: test multiple components working together (database, cache). E2E tests: test the full system from user perspective (slowest, most realistic).

Q: How does Python's `import` system work?

> Python searches `sys.path` (list of directories) in order. First finds built-ins, then installed packages, then local files. `__init__.py` makes a directory a package. `importlib` gives programmatic control.

Q: What is a `__slots__`?

```python
```

```python
class Point:
    __slots__ = ["x", "y"]   # restrict attributes, save memory
    def __init__(self, x, y):
        self.x = x
        self.y = y

# ~3x less memory than regular class
# Cannot add new attributes at runtime
```

Q: What are the SOLID principles in Python?

> S — Single Responsibility: one class, one job. O — Open/Closed: open for extension, closed for modification (use inheritance/composition). L — Liskov Substitution: subclasses must be substitutable for parent. I — Interface Segregation: many specific interfaces better than one big one. D — Dependency Inversion: depend on abstractions, not concretions (use dependency injection).

🚀 Quick Reference Cheatsheet

python

```python
# ──────── MOST-USED BUILT-IN FUNCTIONS ────────
len(x)          # length
range(n)        # 0 to n-1
enumerate(x)    # (index, value) pairs
zip(a, b)       # combine iterables
map(f, x)       # apply f to each element
filter(f, x)    # keep elements where f returns True
sorted(x)       # return sorted copy
reversed(x)     # reverse iterator
sum(x)          # sum of elements
min(x), max(x)  # min/max element
abs(x)          # absolute value
round(x, n)     # round to n decimal places
isinstance(x, T)# check type
type(x)         # get type
id(x)           # memory address (for 'is' comparisons)
hash(x)         # hash value (immutable objects only)
dir(x)          # list attributes/methods
help(x)         # show documentation
vars(x)         # object's __dict__
getattr(obj, "attr", default)   # get attribute safely
setattr(obj, "attr", value)     # set attribute dynamically
hasattr(obj, "attr")            # check if attribute exists
callable(x)     # check if x can be called as function
any(iterable)   # True if any element is True
all(iterable)   # True if all elements are True

# ──────── ONE-LINERS ────────
# Flatten nested list
flat = [x for sub in nested for x in sub]

# Swap variables
a, b = b, a

# Count chars in string
char_count = {c: s.count(c) for c in set(s)}

# Merge and deduplicate two lists preserving order
merged = list(dict.fromkeys(list1 + list2))

# Find most common element
most_common = max(set(lst), key=lst.count)
```

```
# Group list into chunks of n
chunks = [lst[i:i+n] for i in range(0, len(lst), n)]

# Reverse dict
reversed_dict = {v: k for k, v in original_dict.items()}
```

📖 Recommended Learning Path

1. **Week 1-2:** Work through CS50P — all problem sets
2. **Week 3-4:** Build a CLI app (todo list, expense tracker) — apply OOP
3. **Week 4-5:** Build a REST API with FastAPI — applies everything
4. **Week 5-6:** Read Real Python articles on topics you're weak on
5. **Week 6-7:** Write tests for your projects with pytest — aim for 80%+ coverage
6. **Week 7-8:** Read *Fluent Python* Ch. 1-10 — deepens Pythonic understanding
7. **Week 8+:** Contribute to open source, or build a portfolio project

---

> **Final Tip:** The best way to learn Python is to build real projects. Pick something you care about, break it, debug it, improve it. Reading and doing are both essential — but doing is where it sticks.